Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems

Alvaro Rivera

2026-05-26

Abstract

This paper is an analytic theory contribution in the sense of Gregor (2006, *theory for analyzing*, Type I): it introduces a new construct (*porosity*) and an analytic frame (the encoding decision underneath four surface manifestations), and presents one instantiation as existence proof that the inverse encoding is realizable in production. Quantitative evaluation of the instantiation is deferred to a companion case-study paper; the present paper's contribution is the analytic frame, not artifact evaluation. Four bodies of literature — relational database theory, domain-driven design, REST API contract design, and Event Sourcing — independently document a recurring representational defect: schemas that mix unrelated concepts, fields populated for some rows but not others, contracts overloading heterogeneous use cases behind optional parameters, event payloads recording what was sent rather than what was used. Each tradition names the defect locally and prescribes a local remedy. This paper argues that the four are surface manifestations of a single architectural decision — *structure-centric encoding*, the choice to make the representation primary the type-space the form must serve rather than the operation that determined the specific case. We name the defect *porosity* and its principled inversion (operation-centric encoding cross-layer) *anti-porosity*, and identify the three simultaneous conditions a CQRS + Actor Model + Event Sourcing combination must satisfy to sustain it. A system descending from a 2005 wrapper, refined across multiple projects and ported from Java to .NET, has run continuously in production since 2018 satisfying these conditions; that system, Puppeteer, is presented as one realization, not as the foundation of the claim.

Anti-porous architecture

TL;DR

State-oriented persistence and operation-oriented persistence are not merely different storage strategies; they encode fundamentally different representational primaries. The same encoding choice — *what the representation is shaped to record* — recurs at the domain layer (the in-memory type hierarchy) and at the endpoint contract (the wire DTO). When the choice is structure, the system pays the cost in sparsity; when the choice is operation, the system pays it nowhere because the form is sized to the instance that produced it.

Architectures that shape their domain models to fit a storage substrate produce *porous* designs under structural pressure: rich object graphs — recursive, heterogeneous, with different node and edge types — do not survive a relational schema, so domain classes either fragment across joined tables that return empty cells or collapse into wide tables of redundant primitives; endpoint DTOs accumulate optional fields to serve heterogeneous use groups behind a single contract; and persistence layers record *what state exists* without recording *how it came to be*.

This paper argues that these are surface manifestations of a single architectural decision — *structure-centric encoding*, the choice to make the representation primary the type-space the form must serve rather than the operation that determined the specific case. Four bodies of literature (database theory, domain-driven design, REST API design, Event Sourcing) have each diagnosed a face of this decision locally, in their own surface vocabulary, without naming the encoding decision behind it. We name the defect *porosity* and its principled inversion (operation-centric encoding cross-layer) *anti-porosity*, and we identify three simultaneous conditions a CQRS + Actor Model + Event Sourcing combination must satisfy to sustain it. A system satisfying these conditions exists; it is presented in §4 as one realization, not as the foundation of the claim.

Claims this paper makes

1. **Porosity has three distinct surface manifestations, all instances of structure-centric encoding.**
    - *Domain*: the type hierarchy is recorded by its structural envelope — one table covering the universe of variants — rather than by the operation that produced the specific variant in each row. Two visible

patterns: over-normalized schemas, where classes fragment across many tables joined by type and queries return rows of mostly-empty cells; and under-normalized schemas, where classes collapse into wide tables of optional fields conditioned on a `status` column.
- *Endpoint*: a single API contract serving heterogeneous use groups records the union of fields any use case might require; each invocation expresses one case and populates only its slots. The contract is shaped by the type-space of use cases, not by the case the invocation actually expressed.
- *Persistence*: representations capture *what state exists* (horizontal: the structural envelope of values) without capturing *how it came to be* (vertical: the operations that produced them). Three substrate-specific flavors: state-only recording in relational stores (vertical sparsity — operations discarded); fabricated edges in document stores when graph shape exceeds tree shape; and command-DTO-bloat in conventionally serialized event sourcing, where the event mirrors the input payload received rather than the script the execution actually ran.

2. **The three are not independent problems.** Each is a downstream consequence of the same upstream choice: structure as the representational primary at each layer. Once that choice is in place, every local fix recreates the pressure in a different layer.

3. **A combined CQRS + Actor Model + Event Sourcing implementation can reject all three porosities by making operation the representational primary at each layer.**

    - *Domain*: one class per role — one variant per operation outcome — with sum-type discrimination supported natively. The form a row takes is determined by the operation that produced it, not by the union of possible statuses.
    - *Endpoint*: DTOs with filterable optional parameters; the same contract serves heterogeneous use groups but each invocation selects only the slots its case expresses.
    - *Persistence*: dense journals of *verbs* rather than state. Each entry records the script that executed and exactly the inputs that script consumed — not the type-space of inputs the command DTO could accept, not the structural envelope of state at a point in time. The state, however rich, is reconstructed deterministically by replay.
    - The three mechanisms project the same encoding decision (*operation, not envelope*) to three layers.

4. **The inverse encoding is realizable in production; existence is proof of construction, not of principle.** A system whose architecture jointly satisfies the three conditions of Claim 3 has run continuously in production since 2018. It descends from a 2005 wrapper through a sequence of internal projects, was ported from Java to .NET, and currently oper-

ates in the core of eCommerce and payments platforms — payment hubs, account-balance ledgers, KYC pipelines, customer-facing storefronts, and payment-processor integrations. The system is the Puppeteer framework; §4 describes how it realizes each condition (with §4.0 covering the historical genealogy). Because the construct *porosity* was named retrospectively, the system's continued operation demonstrates that the three conditions are jointly satisfiable in production — not that the analytic frame explains the system's success. Appendix B applies the diagnostic to a public schema constructed independently of the present authors. Code references throughout (Appendix A) point to the public repositories; comprehensive quantitative results are deferred to a companion case-study paper.

5. **The contribution is identifying the unified encoding decision behind four locally-named defects.** Four bodies of work — relational database theory, domain-driven design, REST API contract design, and Event Sourcing — each documented surface manifestations of structure-centric encoding in their own vocabulary, without naming the encoding decision itself. CQRS, the Actor Model, and Event Sourcing are individually well-known constituent patterns; what we claim as novel is neither the patterns nor the detection of any single porosity, but the observation that the four traditions converge on a single architectural choice (structure as the representational primary), that *anti-porosity* names its principled inversion (operation as the representational primary), and that the inversion is implementable rather than merely conceivable. We note explicitly that other actor frameworks (Akka, Microsoft Orleans, Proto.Actor, EventStore) could *in principle* be extended to satisfy the three conditions, but none currently does (§7); the analytic frame's independence from any specific realization is therefore *conjectural*, not established. Puppeteer is currently the only realization that jointly satisfies the three conditions, and the open invitation in §5 is the explicit mechanism by which the conjecture of broader realizability would be tested.

6. **CQRS + Actor + Event Sourcing is interesting here for operation-centric encoding cross-layer, not for separation of concerns.** These three patterns are conventionally introduced as separations: reads from writes (CQRS), concurrent units (Actor Model), events from projections (Event Sourcing). When applied together as a single discipline, they constitute a mechanism that projects operation-centric encoding to all three layers in which porosity manifests. This interpretation is available only once the encoding decision is named — porosity as the structure-centric default, anti-porosity as its inversion — rather than treating each layer's symptom as a local problem.

When NOT to use this approach

The principle of anti-porosity has limits, and the implementation pattern that realizes it has narrower limits still. We name two regimes in which the imple-

mentation is not the right fit. In each case, anti-porosity may continue to apply as a diagnostic — porosity remains a real pathology — but the CQRS + Actor + Event Sourcing realization, as instantiated in Puppeteer, imposes costs that may not be repaid.

A. Domains where the actor cannot achieve state locality

The actor model with journal-based persistence assumes that the working set of events required to rehydrate the actor's current state can be bounded. This holds when the actor's automaton passes through cyclic states — created, modifiable, finalized, archived — such that, at some point in the cycle, prior events become safely evictable without altering the current state. Domains where the actor's lifecycle has no such evictable point produce *unbounded rehydration*: every load replays a history that grows without ceiling.

Consider an airline flight. From the moment a schedule is published, an actor we may call `FlightOperation` accumulates events: the first booking, subsequent bookings, seat assignments, gate changes, boarding events, departure, landing. Throughout this sequence the state is genuinely active — answering *"is the flight bookable?"*, *"who is aboard?"*, *"has it departed?"* requires recent history. But once the flight has landed and reconciliation completes, the active state collapses: the flight no longer changes, and the remaining consumers are analytical (capacity-utilization reports, for instance) and can be served from periodic projections. At that point the entire history of bookings and gate changes can be marked evicted without compromising correctness. The actor has reached locality: its working set is bounded.

Compare this to an `Airline` actor that consolidates every flight, every passenger, every booking the airline has ever handled. Such an actor never reaches a "done" state. Its working set grows with operational history, and each rehydration replays material that has no bearing on any decision currently being made. The two designs share the same business domain; they differ in whether the actor's responsibility has a natural end.

The structural analogy with virtual memory is exact. In an operating system, a process exhibits *locality of reference* when its accessed pages stay within a bounded working set; the OS evicts pages outside that set without affecting correctness. When locality is lost — when references span the full address space uniformly — the system enters *thrashing*: pages are reloaded as fast as they are evicted, and useful work approaches zero. The actor model with a journal exhibits the same structural property: events take the role of pages, the actor's required history takes the role of the working set, and skips (event eviction) take the role of page replacement.

Virtual Memory	Actor + Journal
Working set bounded	Non-skipped events bounded
Locality of reference	Locality of automaton state

Virtual Memory	Actor + Journal
Page eviction	Skip / event eviction
Page size	Actor granularity
Thrashing	Unbounded rehydration

The locality property of an actor is, to a substantial degree, downstream of its naming. The word *actor* evokes *action*: a `Packer`, a `Dispatcher`, a `FlightOperation` — names that denote bounded responsibility with a natural end. Names that denote aggregations (*Inventory*, *Catalog*, *Customer*) imply unbounded responsibility, with no point at which prior events become evictable. A noun-aggregator actor is the structural equivalent of an OOP god class: state accumulates because nothing about the name suggests when it should stop. The same observation applies one level further down, to the partitioning of instances: an actor scoped per-country rather than per-shipment reproduces the problem even when the type is well-named. Granularity and partitioning are not separate decisions from naming; they are the same decision expressed at different levels.

Microsoft Orleans makes this granularity choice explicit in the very name of its activations — *grains* — emphasizing each as a small, self-contained unit. The naming heuristic developed here is consistent with that view and offers a verifiable test: if the proposed actor name is a noun-aggregator rather than a verb-doer, the design will likely fail at locality.

In typical production deployments, the cost of locality failure is partly amortized. Continuous-running datacenters confine rehydration to rare events — process restart, hardware failover — rather than steady-state operation; zero-downtime release patterns (developed in Paper 3) hide cold-start latency behind warm replicas during deployment; and journal storage is operationally cheap: it is economically negligible at relevant scales, and its access pattern — sequential append, with occasional sequential reads at rehydration — does not impose the random-I/O performance demands that drive RDBMS disk specifications. A Puppeteer deployment is largely indifferent to disk speed, where a comparable RDBMS workload is bound by it. Locality is therefore best understood as a structural assumption of the model rather than a property whose violation is immediately catastrophic. The "when not to use" case bites when locality fails **and** the amortizing properties above do not hold — for instance, single-instance systems with frequent restarts, or workloads where actor cardinality is so high that eviction-and-reload occurs as part of normal operation.

The mechanism by which Puppeteer evicts journal events — the *skip* — is treated in detail in a companion paper (Paper 4, on zero-downtime deployment via journal-based state handoff). For present purposes it suffices to note that skips presuppose locality: the framework can implement skips, but it cannot

manufacture locality where the actor's responsibility is unbounded. Locality is a property of the design, not of the framework.

B. Domains where the actor's verbs cannot be kept fast

Each actor processes its mailbox serially: only one verb executes at a time on a given actor instance. The framework sustains thousands of requests per second per actor, at peak, on the assumption that every verb completes within a few milliseconds. A verb that blocks for hundreds of milliseconds — synchronous I/O to a slow service, an expensive query, a costly computation — does not merely slow itself; it blocks every subsequent verb in the mailbox, and ingest throughput collapses. The single-thread invariant of the actor turns from a correctness guarantee into a throughput cap.

Two patterns sustain the fast-verb invariant in practice. **I/O hoisting**: the caller resolves external dependencies — third-party calls, slow lookups, expensive joins — before invoking `perform`, and passes the resolved values as parameters. The actor receives data; it does not fetch. **Saga-phased I/O**: when the workflow genuinely requires interleaved external calls, the work is decomposed into a Saga. The actor advances one phase, releases, an external coordinator performs the I/O, and the response re-enters the actor as the next event. The actor never waits inside a single verb.

Where neither pattern applies — synchronous I/O is intrinsic to the verb, or the underlying algorithm is unavoidably slow — the actor's serial processing is structurally incompatible with the workload. In those cases the framework's claim of high per-actor throughput cannot be honored.

In both regimes, the principle of anti-porosity may continue to apply as a diagnostic; what fails is the realization, not the principle.

---

1. Introduction: the invisible cost of porous designs

This paper makes an analytic theory contribution. It identifies and names an encoding decision that prior literature has documented separately in surface vocabulary across four traditions (the construct, *porosity*); derives the design principle required to invert the decision across the three layers in which it manifests (*anti-porosity*); and presents an instantiation — a system whose underlying observations date to a 2005 prototype and whose current form has been in continuous production use since 2018 — as confirmation that the inversion is realizable. The contribution is the analytic frame; the instantiation is an existence proof of realizability. The genre is *theory for analyzing* (Gregor, 2006, Type I): analytic constructs that allow phenomena to be described, classified, and unified, with empirical evaluation supplementary. Hevner-style design science evaluation (Hevner, March, Park, & Ram, 2004) — which would require bundling artifact evaluation with construct introduction — is deferred to a companion

case-study paper; the Gregor Type I frame separates the two cleanly. The paper is self-contained as a Gregor Type I analytic contribution: the frame stands on independent grounds and can be evaluated as such. Its position relative to companion papers in a larger series is additional context, not a precondition for its evaluation.

A defect recurs in mainstream software architectures: representations that carry more structure than the operations they describe ever populate. Relational schemas accumulate columns whose values depend on the row's state — required for some, irrelevant for others. API contracts grow optional fields that serve different use groups under one route, with no contractual signal of which fields belong to which case. Event payloads serialize whatever the command DTO carried, regardless of which inputs the operation consumed. Each looks local, and each has a literature that diagnoses it locally: database theory names normalization anomalies and NULL semantics; domain-driven design names anemic and persistence-driven models; REST API design names overloaded contracts and field bloat; Event Sourcing names payload bloat and command-event coupling. Four traditions, four vocabularies, four sets of remedies. A partial cross-tradition unification has long existed under the name *object-relational impedance mismatch* (Ireland, Bowers, Newton, & Waugh, 2009), but it covers a single boundary — objects against relational stores — and treats the cost as a problem of mapping rather than as a consequence of an encoding decision. To our knowledge, no prior work names the encoding decision underneath the four surface manifestations at the level of generality this paper proposes; §7 reviews the prior unifications and positions the contribution as extension rather than discovery.

We call this defect *porosity*: the representation is shaped to record structure (the type-space the form must accept) rather than the operation that determined the specific case. Its visible symptom is widespread structural emptiness — fields without values, columns whose meaning depends on the row, queries returning rows with cells that the application never reads, plus the absence of any record of how state came to be where state-only persistence is used. The defect emerges most visibly when complex or polymorphic abstractions are forced into relational tables (the historical default of structure-centric encoding), and intensifies as the abstractions become richer; it recurs at every layer where structure is treated as the representational primary, including endpoint contracts and event-sourcing payloads independent of any substrate. The vocabulary that names the encoding decision at the level of generality needed to see it across surfaces — beyond the object/relational boundary that impedance-mismatch literature already covers — has not, to our knowledge, been established in prior work. Developers do not say *this design is porous*; they say *this design is fine; the empty fields are normal.* Decades of structure-first modeling have made the trade-offs that produce porosity (over-normalization versus under-normalization, joins versus wide rows, NULLs versus duplicate tables, command-DTO-as-event versus per-call script) feel like inherent properties of software construction. They are not. They are the consequence of one architectural decision — structure as the rep-

resentational primary — repeated millions of times across surfaces. Naming the decision is the first step toward seeing it.

This paper argues that porosity is not inevitable, that the four traditions cited above describe surface manifestations of a single encoding decision (structure as the representational primary), and that their independent local remedies are partial solutions to a problem that admits a unified one — the inverse decision applied across surfaces. We name the unified rejection of porosity *anti-porosity*, identify the conditions under which a CQRS + Actor Model + Event Sourcing combination sustains it across all three layers in which it manifests (domain, endpoint, persistence), and present a system that satisfies those conditions as confirmation that the principle is realizable. The historical genealogy of that system — how it came to satisfy the conditions before they were named — is presented in §4.0, alongside the mechanisms it uses to do so.

The paper is organized as follows. §2 enumerates three layers of porosity and argues that each is a face of the same defect. §3 introduces anti-porosity as a unified design principle and states the conditions a system must satisfy to realize it. §4 describes the origins and mechanisms by which one such system, Puppeteer, realizes those conditions; the section is descriptive of an instantiation, not constitutive of the principle. §5 reports operational observations drawn from production deployments. §6 addresses anticipated counter-arguments from the principle. §7 places this work in the context of related literature. §8 concludes.

1.1 A formal characterization

Porosity is not a database problem, nor an API problem, nor an event-sourcing problem. It is a defect of the representational primary — the architectural choice of what the representation is shaped to record. Four formal traditions, each diagnosing the defect from its own surface, converge on the same encoding decision.

The defect, stated in one sentence:

> *Porosity is the property of a representation that encodes* ***structure*** *(a schema, contract, payload shape, or state form) when encoding the* ***operation*** *that determined the specific case would preserve what that case actually contributed. Structure is* ***type-spaced****: the form covers the universe of cases the representation must serve. Operation is* ***instance-spaced****: the case that this invocation, this row, or this execution produced. Where the representation is shaped for the type-space, capacity that the instance does not use is paid in sparsity.*

The dual:

> *Anti-porosity, or density preservation, is the choice to make the operation the representational primary, encoded at each layer as an instance-shaped artifact — the* script* at the persistence layer, the sum-type *variant* at the domain layer, the selected *use case* at the

endpoint layer — and to derive structural projections only where downstream consumption requires them.*

The defect has two visible forms. **Horizontal sparsity**: slots populated only by some instances of a shared form (NULL columns by status, optional DTO fields by use case, ES payload fields the execution did not consume). **Vertical sparsity**: the operational trace that the operation determined but the form did not capture (state schemas that record terminal values without the operations that produced them). Both are symptoms of the same encoding decision — structure as primary.

The four formalisms that follow each illustrate this defect in their own vocabulary; their convergence is on the encoding decision, not on a single surface.

Graph theory. Domain models can be formalized as *semantic graphs*: nodes typed by entity class, edges typed by relation, with both nodes and edges potentially heterogeneous in type (Diestel, 2017). The graph is itself an operational object — edges express verbs that relate nodes — but conventional relational projection captures only the nodes' structural attributes, discarding the operational character of the edges. The graph-database and Semantic Web literatures (Lassila & Swick, 1999; Robinson et al., 2015) preserve graph shape directly; the relational literature does not.

Information theory. This paper's terminology — *structural capacity* (bits available to encode the type-space of cases the form admits) and *informational content* (bits the specific instance actually contributes) — takes its inspiration from, but is not derived formally from, Shannon's framework (Shannon, 1948). The lineage is explicit; the derivation is not. Shannon's *entropy* and *channel capacity* are defined under assumptions (equiprobable states, channel noise as transmission perturbation) we do not establish here; the terms above are introduced to characterize representational shape, not channel behavior. Structure-centric encoding fixes the form to cover the type-space; the instance's contribution occupies a subset of that capacity, leaving the remainder as structural excess.

Storage engines. Each storage substrate exposes what we will call its *structural alphabet*: the set of shapes it can natively express. The term is introduced here; survey treatments (Hellerstein, Stonebraker, & Hamilton, 2007) catalogue substrates and their internals without isolating this property as an abstraction. Relational stores express rows × columns × types; document stores express nested trees of typed values; graph stores express labeled directed graphs. These alphabets are all *structural* — they capture forms that data can take, not operations that produce data. An operation-centric substrate exposes a different alphabet: the primary unit is verb-instance (an executable script), not row-shape. The journal substrate developed in §4.3 is one such case.

Database normalization theory. Codd's relational model (Codd, 1970), refined through successive normal forms (Codd, 1972; Fagin, 1977), is the canonical historical formalism for representing data in the relational alphabet. Nor-

malization is prescribed to eliminate update, insert, and delete anomalies. The anomalies are themselves diagnostic of structure-centric encoding: they arise because the relational model records state-as-form without the operations that produced it. The procedures that resolve them — splitting wide tables, introducing join columns, accepting NULLs in optional attributes — work within the structure-centric paradigm; they do not question the encoding decision.

Synthesis. Each tradition saw the defect from its own layer and named a local remedy. The unification proposed here is not over four symptoms but over one decision: across substrates and surfaces, the representational primary was chosen as structure rather than operation. Once the encoding decision is named, the four traditions become four windows onto the same defect.

The vocabulary of the definitions above is partly borrowed and partly introduced: *semantic graph* from graph theory; *sparsity* generalizes the information-theoretic notion; *structural alphabet* and *structural capacity / informational content* are this paper's terms; *normal forms* and *anomalies* from database theory. The contribution is the encoding-decision frame — naming the architectural choice (structure-as-primary) that the four traditions each documented in their own surface vocabulary without naming as a single decision.

1.2 Terminology

This paper uses several near-synonyms to refer to aspects of the same conceptual unit. We distinguish them once and use them consistently.

- *Operation*: the conceptual unit — a verb applied to its consumed inputs. This is the artifact treated as the representational primary under operation-centric encoding.
- *Verb*: the action body defined once in the domain library (a method or procedure with declared parameters). The same verb is invoked many times with different inputs.
- *Invocation*: the act of applying a verb to specific inputs at a specific time.
- *Script*: the textual representation of an invocation — the form persisted in the executable journal (§4.3). A script is the program-form of an operation.
- *Execution*: the runtime evaluation of the script against the current domain library, producing state.
- *Event*: in *conventional event sourcing*, the serialized DTO record produced after an invocation. This paper reserves *event* for the conventional ES usage and uses *script* or *operation* for the operation-centric form; the distinction is foundational (§2.3, §4.3) and not a naming preference.

The terms describe the same lifecycle phenomenon viewed from different vantage points: the verb is defined once, an invocation occurs, the script captures it, the execution produces state. The persisted artifact under operation-centric encoding is the script; the persisted artifact under conventional event sourcing is the event.

2. Three faces of porosity

2.1 Porosity in the domain layer

The domain layer's primary structural artifact is the type hierarchy. Under structure-centric encoding, that hierarchy is projected onto a homogeneous form — a table — whose shape covers the type-space (all possible variants); each row occupies only one variant. The sparsity that follows is the cost of recording structure-of-the-class rather than the operation that produced the row.

In object-oriented modeling, the canonical pattern for representing entities that pass through distinct lifecycle states is inheritance: each state is a subtype, with its own fields, invariants, and methods. A purchase order, for example, naturally decomposes into `Draft`, `Requested`, `Paid`, `Dispatched` — each subtype carrying only the attributes and operations that its state admits. The transition from one state to another is the construction of a new subtype instance, not the mutation of fields on a single class.

The relational model does not support this pattern cleanly. A type hierarchy can be projected onto a relational schema via one of three known idioms — single-table, class-table, or concrete-table inheritance (Fowler, 2002) — each of which produces porosity in a different form. In practice, single-table inheritance dominates: the schema collapses the four subtypes into one wide table with a `status` column and optional fields populated only for some rows. Attributes that belong logically to `Paid` are nullable for rows still in `Draft`; attributes that belong to `Dispatched` are nullable for everything else. The hierarchy is recovered at read time by inspecting the `status` field and conditionally interpreting the rest. The semantic graph — a clean polymorphic structure — has been projected onto a homogeneous vector space, and the gap between them surfaces as NULLs.

Formally, the relational projection encodes a sum type as a product type. The state hierarchy — `Draft`, `Requested`, `Paid`, `Dispatched`, each variant with its own fields and invariants — is naturally a tagged union: a sum type. The relational schema, lacking native sum-type support, encodes it as a product type — a single tuple of fields that must coexist regardless of variant, with the discriminator demoted to a column. The mismatch is structural. In the information-theoretic vocabulary of §1.1, sparsity — structural symbols present in the representation that carry no information for any specific instance — is the inevitable cost of forcing a sum-type structure through a product-type alphabet.

The choice is not driven by ignorance of the alternative. Developers familiar with object-oriented design recognize that inheritance is the structurally cleaner pattern; they adopt the property-field approach because the substrate makes it the path of least resistance. The other two idioms impose joins by type, schema duplication, and migration overhead severe enough to outweigh the modeling benefit. Porosity here is the signature of a forced compromise — not an absence of OOP wisdom, but a substrate that does not admit it.

The downstream effects extend beyond schema shape. State transitions on the porous table become updates against a row that other transactions may also read or modify; lock contention emerges as a structural cost of the design choice, not as an accident of high traffic. The broader observation — that a domain whose lifecycle was never logically concurrent now pays for transactional concurrency it did not request — is treated separately, as a distinct symptom of persistence-as-source-of-truth, in Paper 5.

2.2 Porosity in the endpoint layer

The endpoint layer's primary contractual artifact is the DTO — a fixed-shape tuple of fields defining what flows between client and server. The same structure-vs-operation tension reappears one layer up: the DTO is shaped for the type-space of use cases the endpoint must serve, while each invocation expresses one specific case.

When an endpoint serves a single, homogeneous use case, the DTO is well-defined and dense: each field participates in the operation's semantics. When an endpoint serves heterogeneous use groups behind a single route, the DTO accumulates optional fields covering the union of all cases — the contract is shaped by the universe, not by the case. This mirrors the wide table with `status`-conditioned columns of §2.1, with the encoding decision identical: structure as primary at the contract layer.

From the perspective of type theory, a DTO is typically modeled as a product type: a fixed collection of fields that must coexist. Heterogeneous use groups, by contrast, correspond naturally to a sum type — a tagged union — where each variant carries only the fields relevant to its case. When a product type is forced to encode what is semantically a sum type, sparsity appears as optional fields and conditional validation.

Consider an endpoint `POST /orders` that serves both a customer self-checkout flow and an administrative batch-insertion flow. The customer flow specifies items, a delivery address, and a payment token; the administrative flow specifies override pricing, source attribution, and an internal correlation ID. A unified DTO carries the union of all fields, validates conditionally based on caller identity or feature flags, and provides no contract-level indication of which fields belong to which use group. Documentation shoulders the burden the schema cannot.

Formally, this is again a projection of heterogeneous semantic variants onto a homogeneous vector space. In terms of information theory, the DTO's structural capacity exceeds the information actually conveyed by any single call. The unused fields constitute representational sparsity: structural symbols present in the representation that carry no information for the specific operation being performed.

The defect manifests symmetrically on the response side. The same DTO that

returns an order's full detail to a desktop client returns more than a mobile client requires; clients either over-fetch or fragment the operation into multiple round-trips, neither of which the original fixed-shape schema admits. The DTO, as a homogeneous projection of underlying semantic variants, reproduces at the wire contract level the porosity already observed in relational schemas of §2.1.

Costs accumulate at multiple levels. Validation logic becomes conditional. DTO families proliferate as variants are introduced to handle slightly different use groups. Client code absorbs out-of-band knowledge about which fields apply when. The contract — intended to be the disciplined surface between systems — becomes porous: its meaning depends on contextual knowledge that the contract itself does not encode.

2.3 Porosity in the persistence layer

The persistence layer's job is to durably capture what the system has done. Three regimes recur in practice — relational stores, document stores, and conventionally implemented event sourcing — and each manifests porosity differently while sharing a common cause: each chose structure as the representational primary. Relational and document stores record state (the structure of the world at a point in time), discarding the operations that produced it — vertical sparsity. Conventionally implemented event sourcing improves on state-only persistence by recording verbs, but persists the command DTO (the structure of what was received) rather than the script of what executed — horizontal sparsity, of a different shape from §2.1 but the same encoding decision.

Relational persistence captures state, not the operations that produced it. A row records that an order is in `paid` state and stores the amount and payment method, but cannot answer how the order arrived there or what triggered the transition. The information loss is structural and asymmetric: state can be derived from operations by replay, but operations cannot be derived from state — the function from operations to state is many-to-one. Recording only state is therefore strictly less informative than recording operations; under evolved semantics (a bug fix, a richer derivation, a new derived field) or under audit (who triggered what, in what order), the missing direction is the one that matters and the one the schema cannot supply. The state-conditioned fields of §2.1 propagate to persistence: when invariants depend on state — for instance, `amount` is required only when `status = paid` — the schema cannot enforce them. The available alphabet (foreign keys, nullability, primitive `CHECK`) admits only primitive referential integrity, never the conditional, cross-field, cross-object invariants that emerge naturally from a sum-type domain. Invariant enforcement migrates to code by structural necessity, not by stylistic choice.

Document stores partially relieve the schema rigidity by allowing nested heterogeneous shapes. A purchase order can be a document with embedded items, addresses, and payment details, sparing the joins that relational schemas require. But the alphabet admits only a particular kind of structure: the labeled

rooted tree. Domain semantic graphs are not constrained to trees — they admit cycles, references shared across documents, and edges of arbitrary type. When the same item is referenced from multiple orders, or when payments link to other documents, the missing edges must be fabricated at the application layer. The substrate handles trees natively and degrades the moment graph shape is required, in the strict graph-theoretic sense of §1.1.

Event sourcing, in its mainstream conception, improves on both by recording operations rather than state: replay reconstructs state by re-applying events. But in its conventional form — where events are serialized data structures mirroring the input commands — the entire input payload is captured regardless of which inputs the operation actually consumed. Consider an operation `RecordPayment` whose interface accepts twenty parameters; internally, the state-changing logic consumes four. The conventional implementation persists all twenty in the resulting event. The event payload is a fixed-shape tuple; the unconsumed fields constitute, in the informal sense established in §1.1, structural excess alongside the consumed content. Event sourcing inherits the porosity it was intended to escape: the journal records what the operation *received*, not what it *consumed*. The distinction is itself foundational and not merely a serialization preference. *Conventional event sourcing stores events (serialized DTOs); operation-centric persistence stores scripts (executable invocations).* They are not interchangeable forms of the same data — they are different representational primaries, and the structure of each (received-DTO shape vs. consumed-input shape) is itself a face of the encoding decision §1.1 names.

The persistence layer thus exhibits porosity in three forms: state without operations in relational stores (vertical sparsity), fabricated edges where graph shape exceeds tree shape in document stores (horizontal sparsity of structural mismatch), and command-DTO-as-event in conventionally serialized event sourcing (horizontal sparsity of received-not-consumed). The encoding decision is identical across the three: structure — state, tree, or command DTO — is treated as the representational primary, and the operation that produced it (or the script that executed) is either discarded or recreated as structural envelope. Three layers, one decision: §3 establishes the principle that addresses them as one.

3. The unified principle: anti-porosity

A system is anti-porous when its representations, at every layer, are *operation-centric*: shaped by the instance the operation determined rather than by the type-space the form might serve. In the vocabulary established in §1.1, this means choosing operation as the representational primary — encoding the script that executed (verb + consumed inputs), the variant the operation produced (sum-type instance), or the use case the invocation expressed (per-call wire selectivity) — rather than the structural envelope into which any instance of the type must fit. In the language of type theory, this means representing heterogeneity as sum types rather than product types; in the language of information theory, it means structural capacity sized to the instance's contribution rather

than to the type-space.

A criterion of demarcation. The structure-vs-operation encoding decision is observable wherever a representation is shaped for a universe of cases rather than the specific instance: function signatures with unused optional parameters, Protocol Buffer schemas with many `optional` fields, HTML forms serving heterogeneous use groups, generic-type parameters never specialized to specific types. The present paper draws a narrower line: it analyzes layers that persist artifacts whose lifetime exceeds the operation that wrote them — the in-memory aggregate (the domain), the wire DTO at the contract boundary (the endpoint), and the durable schema or journal (persistence). Representations whose lifetime is bounded by a single operation — a function call's argument list, a generic type parameter at one instantiation, an HTML form rendered for one request — exhibit the same encoding decision but do not pay the cost in *stored* sparsity. The frame may apply to them with appropriate adjustment; the present paper does not claim it across them.

The principle manifests as three simultaneous conditions, each inverting the structure-centric default observed in §2. First, the domain layer admits sum-type structure natively: a state hierarchy is encoded as variants (one per operation outcome), not as a product type with a `status` discriminator and conditionally populated columns. Second, the endpoint layer admits per-call selectivity at the wire: contracts express sum-type discrimination across heterogeneous use groups (one variant per invocation), not a fixed-shape DTO that unions all of them. Third, the persistence layer records operations and only the inputs the operations actually consumed: the journal carries the script that executed (an instance-shaped artifact), not the structural envelope of what was received.

The three conditions are not independent fixes; they are the consequence of a single decision applied at three surfaces. As §2 made structurally evident, each manifestation of porosity arises from the same architectural choice — to treat structure as the representational primary, the type-space as the form, and the instance's contribution as a subset that leaves the remainder sparse. Anti-porosity is the inverse choice — to treat operation as the representational primary — applied as a single discipline across the three layers. A patch applied only at the domain (aggressive denormalization, hand-rolled inheritance) leaves the API and journal demanding structural envelopes; a patch applied only at persistence (compact event encoding) leaves the journal's density at the mercy of upstream layers' choices. Anti-porosity is not the sum of three local choices; it is a single architectural decision projected to three surfaces.

Operationally, anti-porosity is *density preservation*: the representation's structural capacity matches its informational content because the form was sized to the operation's contribution rather than to the type-space — consumed content retained, structural excess discarded. The combination of CQRS, the Actor Model, and Event Sourcing — conventionally presented as three separations of concern — admits a different reading once anti-porosity is named as the principle they jointly satisfy: they constitute, when applied as a single discipline, a

mechanism for density preservation across representations. The principle does not prescribe a particular framework. Akka, Microsoft Orleans, Proto.Actor, or any system that combines actors, CQRS, and event sourcing could in principle satisfy the three conditions; doing so requires a substrate decision none of those frameworks has currently made (§7). The next section describes one realization — the Puppeteer framework — selected as the existence proof for the principle, not as its definition. The realization rests on a single architectural choice: that operations themselves — verbs invoked by callers — be recorded as the durable representation. This extends the principle of *code-as-data* (familiar from Lisp at the language layer) to the persistence layer in a narrower form than Lisp's strong homoiconicity. §4.3 develops the property formally as *script-form persistence*, and the resulting artifact as the *executable journal*.

4. An instantiation: how Puppeteer realizes the three conditions

This section describes how the Puppeteer framework realizes the three conditions stated in §3. It is the only section of this paper in which authorial voice ("we") and the specifics of one implementation are concentrated. The realization is presented as an existence proof for the principle, not as its definition; the conditions could be realized differently in another framework that made comparable substrate decisions (§7).

4.0 Origins of the instantiation

A practical encounter with the underlying phenomenon predates the present analysis by nearly two decades. In 2005, work that eventually became the Puppeteer framework began as a wrapper around a DLL that exposed services in the then-emerging vocabulary of *web services*. While building this wrapper, an unexpected property of the construction emerged. By intercepting the parameters of every incoming call and writing them, in order, to a log — what would now be called a *journal* — the full state of the underlying automaton could be recovered without inspecting its code. Starting from any consistent prior state and replaying the log, the automaton arrived at exactly the same final state. Persistence, traditionally treated as a separately designed concern, had emerged as a side effect of the wrapper. The phenomenon was named *autopersistence*, after its observable effect rather than from any theoretical premise. What the wrapper-log approach made visible was something the relational-persistence approach had hidden: the log was *dense* — it contained no empty fields, recording only what each call had actually used. The relational schemas the same applications wrote to, by contrast, were full of NULLs, columns that changed meaning by row, and joins that returned mostly-empty cells. The journal was *dense*; the schema was *porous*. The label and the principle followed; the observation came first. The mechanisms described in §4.1–§4.3 are the present-day form of properties already implicit in the 2005 wrapper: a domain visible to the framework only by reflection, parameters captured exactly as the call carried

them, and a journal that records operations rather than state. Between the 2005 prototype and the present design, the framework was used in a sequence of internal projects, ported from Java to .NET, and progressively refined; the form described in §4.1–§4.3 has been in continuous production use since 2018.

4.1 Roles, not concepts: the library

Anti-porosity in the domain layer admits a single thesis: **a sum type, not a table**. The framework partitions the domain by *roles* — operations that act — rather than by *entities* — nouns that exist. Each role is a subtype: a class with its own fields, invariants, and verbs. The purchase order example of §2.1 is encoded directly as a sum type, with `Draft`, `Requested`, `Paid`, and `Dispatched` realized as distinct subtypes of a common base, each carrying only the attributes its role admits. The system's domain is the union of these roles, not a flattened table indexed by a `status` column.

The classes that realize these roles form what the framework calls the *library* — pure conceptual artifacts that do not know which "play" they participate in. A `Packer` is a *class puppet*: it knows how to pack; it does not know whether the packer instance is being persisted, replayed, or audited. The framework discovers domain types reflectively at runtime, scanning the configured library assemblies (`Actor.cs:15` declares the marker base class; `DomainLibraries.cs:117-128` performs the assembly scan; `ActorHandler.cs:61` invokes it at handler construction). The domain need not register itself; the framework inspects what is there. The continuity is structural, not anecdotal: the 2005 wrapper (§4.0) intercepted parameters without inspecting the DLL it wrapped; reflection-based discovery preserves that property as a structural feature of the current design.

Polymorphism is a first-class concept of the DSL itself, distinct from how it is realized in the host language. Argument-parameter compatibility is computed at parse time via subtype checks (`AstExpression.cs:236-246`, `AreCompatible`); runtime substitution of subtypes is handled by the symbol table (`SymbolTable.cs:134`, `IsSubclassOf` check). Where the host language (C#) optimizes for verbose precision, the DSL optimizes for script readability and for boundary decoupling. Type promotion at the call site is permissive: polymorphic arrays promote to whatever collection type the library declares (`List`, `IEnumerable`, array, and so on); parameters accept any enumeration and promote at parse time to the specific enum the library expects. The DSL writes `Credit`, the DTO carries `Credit` as text, and the library's `PaymentMethod` enum receives the matching constant — no lexical change at any boundary. The contract is structurally decoupled from the domain class: each side may evolve its enum vocabulary independently, with the DSL mediating. **This is not syntactic sugar but a design decision: the DSL is shaped to read as domain narrative, not as a transcription of a programming language.**

Anti-porosity in the domain layer follows from the alignment of three properties:

role-oriented partitioning (sum-type variants in the type hierarchy), reflection-based discovery (the framework imposes no schema on the library), and DSL-level polymorphism (sum-type discrimination is honored end-to-end in the operation contract). The remaining two layers — endpoint and persistence — are realized by separate but compatible mechanisms, treated in §4.2 and §4.3.

4.2 Parameter modifiers: `?` and `Eval`

The framework makes the direction of every value flow explicit through four parameter modifiers: `In`, `Out`, `InOut`, and `Eval`. The first three mirror the in/out/ref convention familiar from systems languages, ensuring caller-puppet isolation: the puppet cannot modify the caller's environment outside the declared direction. The fourth, `Eval`, is unique to this realization. The modifier system serves two complementary purposes: explicit isolation at the call contract, and honesty in the operation's journal record.

For `Out` parameters, the journal writes the literal character `?` as a placeholder (`Parameters.cs:755-757`). The reason is structural: an `Out` parameter's value is computed by the puppet, not supplied by the caller — there is no input value to record. At replay, the deserializer reads `?` and produces a default value of the parameter's type (`Parameters.cs:780-782`); the puppet's logic re-executes and fills the slot. The journal thus records exactly what the call carried at the input boundary — output slots appear as honest reservations, not fictitious inputs.

`Eval` parameters address a different problem: values the puppet uses that are not supplied by the caller and are not deterministic across replays. Generated identifiers and game-session state — a random number or a gameboard captured for a game's session — are typical cases: values that pertain to the operation's semantics but cannot be re-derived at replay. The `Eval` modifier directs the puppet to capture the value, on first execution, as a literal-assigning script (`name = (type)(value);`), persisted as part of the operation's journal record (`Parameter.cs:33-36, 163-224, 228-247`). On replay, the script re-executes and assigns the same literal — determinism restored. V1's interpreted DSL had an `eval` command for the same purpose; V2 replaced it with the parameter modifier because the command form could not always be compiled (`EvalStatement.cs:52-55`).

The modifier system ensures the journal carries exactly what the call was:

- Caller inputs recorded as values.
- Puppet outputs recorded as `?` reservations.
- Non-deterministic values recorded as literal-capturing scripts.

Anti-porosity at the parameter level emerges from two complementary disciplines:

- Explicit direction at the call boundary.
- Explicit capture of non-determinism.

`?` and `Eval` are dual mechanisms that operationalize anti-porosity at parameter granularity. Integration into the broader journal — and the script-form property that allows replay against an evolved domain — is treated in §4.3.

Boundary cases of `Eval`. Two cases of the `Eval` mechanism are worth naming explicitly, both of which the production deployments cited in §5 have had to address.

External dependencies. When an `Eval` expression resolves to a value derived from external state — a currency exchange rate at the moment of invocation, a foreign-key lookup against another service — the framework captures the resolved literal but not the dependency. The journal entry is `rate = (decimal)510.0`, not `rate = CurrentExchangeRate("USD","CRC")`. On replay against a future library, the captured literal is reproduced exactly; the external state may have changed. This is correct under *snapshot semantics*: the journal records what the system actually saw at the time of the operation, not what the world is now, and replay reconstructs the historical execution. The framework's contract is historical faithfulness, not present truth. Applications that require present-truth on replay (uncommon, since replay is typically used for state reconstruction, audit, or upgrades against evolved library logic) must re-derive the external value explicitly in the new library rather than relying on the captured literal. The boundary case is worth naming because the contract is sometimes assumed to be present truth and the framework's contract is not that.

Sensitive values. `Eval` captures literal values into the journal, persisted as text. For values that should not appear in plaintext at rest — cryptographic nonces, payment tokens, PII — the framework requires deployment-level mitigation. Three mitigations are available and have been used in the production payment deployments cited in §5: (a) at-rest encryption of the journal storage substrate (available in all four shipping backends); (b) application-level redaction policies applied at write time (specific to the application, e.g., card-number tokenization before `Eval` captures the value); (c) in-script transformation so that the `Eval`'d value is already cryptographically opaque before capture (e.g., the verb captures a hash or token, not the secret). The framework does not provide column-level encryption *inside* the script representation, because an encrypted literal in a script would not parse and execute on replay; opaque encrypted blobs must be decrypted before script evaluation, which is the application's responsibility. The point worth naming here is that `Eval`'s literal-capture property does not exempt journal storage from the standard at-rest-encryption disciplines other persistence substrates require. Conventional ORM-style serialization shares this property and the same mitigation playbook; the framework inherits both the property and the obligation.

4.3 Operations, not state: the executable journal

In the framing developed here, the primary property of event sourcing is not temporal traceability but density preservation: it preserves semantic density that state-based persistence models discard.

By storing operations rather than state projections, the journal maintains an isomorphic representation of the semantic graph that tabular or document projections render sparse. The mechanism that achieves this we call *script-form persistence*: each entry in the journal is simultaneously data — storable, indexable bytes — and a program — a parsable, executable script in the framework's DSL. The journal does not store a serialization of the actor's state; it stores the script that produced it. We call the resulting artifact the *executable journal*: a durable log of executable DSL scripts, replayable against the current library to reconstruct state.

The relationship to *homoiconicity* in its strong sense, familiar from Lisp (McCarthy, 1960; the term itself coined by Mooers and Deutsch, 1965), is one of *lineage, not equivalence*. Lisp's homoiconicity has consequences — macros that operate on S-expressions as primitive data structures of the language, eval-quote symmetry, code-modifying-code at the language layer — that the present realization does not claim. The DSL scripts in the executable journal are parsed by the framework parser and executed by the framework runtime; they are not consumed as primitive data structures by other DSL scripts, which is the property that would justify the strong term. The narrower property the present paper claims, and the only one its argument requires, is that persisted journal entries are textual representations of executable programs in the actor's DSL — retrievable as text, parsable into ASTs, executable against the current library. Bash scripts, SQL stored procedures, and shell command logs share this narrower property; we therefore do not claim homoiconicity in the strong sense. *Executable journal* and *script-form persistence* are the precise terms for what the argument requires; the lineage to Lisp is acknowledged without claim of identity.

Conventional event sourcing persists the command DTO; the present realization persists the execution script. The script encodes consumed inputs only — the bloat of received-but-unused parameters does not arise by construction.

The script-form representation enables a capability that state-storing substrates structurally cannot offer: re-execution against new semantics. When the domain library evolves — a logic fix, a richer derivation, a new computation depending on past inputs — the journal replays against the updated library, producing corrected projections without altering the historical record. Storage engines that persist state, by contrast, can only restore the state that was written; the operations that produced it have been discarded.

The density preservation property is structural, not accidental. **A script cannot contain parameters the execution did not use, because the**

script is generated after parameter direction and evaluation rules have been applied (§4.2). The journal therefore inherits density from the modifier discipline: caller inputs recorded as values, puppet outputs as `?` reservations, non-deterministic values as literal-capturing scripts. Two operational properties verify this in code. First, density is preserved through an asymmetry between inputs received and inputs consumed: only the latter are serialized (`Parameters.cs:755-757`, `:780-782`; `Lexer.cs:600`, where `?` is tokenised as `TokenType.question`). Second, the journal admits exactly two primitive operations — append and read-forward (`JournalWriter.cs:65`; `JournalReader.cs:35`) — exposed through a unifying interface (`Diary.cs`); the vocabulary of relational stores — SELECT, UPDATE, DELETE, INSERT, and the transactional locking that surrounds them — is absent by construction, not by convention.

A journal of DSL scripts therefore combines three properties — script-form persistence (executable text retrievable and re-runnable), graph preservation, and density preservation — under one representational substrate.

5. Empirical observations

The empirical claims in this paper are minimal by design, consistent with the *theory for analyzing* genre declared in §1 (Gregor, 2006, Type I): the contribution is the analytic frame (the encoding decision underneath four locally-named manifestations), for which an existence proof of realizability is supplementary. Quantitative measurements of the instantiation are deferred to a companion case-study paper; the analytic frame stands without them. The principle predicts that representations satisfying the three conditions of §3 will exhibit higher information density per entry than relational projections of the same operations; the magnitude of the resulting storage-volume gap depends on the workload regime (broadcast vs pointwise vs analytical), characterized below. The worked example illustrates the broadcast regime where the gap is largest, and the deployed instantiation listed at the end of the section confirms the principle survives in production.

A worked example. Consider a generic domain in which buyers issue purchase orders against future scheduled events; an event is *confirmed* once it has occurred; and an event is *settled* with one of several outcomes (award, refund, restatement). The pattern recurs across ticketing, conditional pre-orders, and milestone escrow. Three primitive verbs realize the lifecycle:

Phase	Statement	Scope
Pre-event (× n)	`order = Company.Purchase(buyer, items, events, amount, currency)`	one order per call; ranges over items × events

Phase	Statement	Scope
Event occurs	`event.Confirm(date, operator)`	one event; implicitly all order-items bound to it
Resolution	`event.Settle(outcome, date, operator)`	one event; implicitly all order-items bound to it

The receiver determines scope; no row enumeration is required.

The journal model. The framework's V2 pattern stores each action's verb body once in an action library; the journal records per invocation only the action identifier, the parameter values, and minimal administrative metadata (timestamp, operator, entry ID). The journal grows with parameters, not with action bodies. (V1's raw-script representation, in which the verbatim script appears in each entry, is preserved for backward compatibility.) This split mirrors the separation between definition and invocation in any compiled or interpreted language; in this realization, that separation extends to persistence.

Storage backends. The conditions of §3 concern the representational shape of journal entries, not the substrate that physically stores them. Four backends ship with the framework — filesystem (UTF-8 script entries in append-only files), in-memory (for testing and ephemeral actors), and two relational engines (MySQL and SQL Server) — and the script-form, density, and append-only access patterns are preserved across all four. The relational backends expose the journal through standard SQL interfaces familiar to operations teams; entries remain executable scripts queryable as text columns. Choice of backend is a deployment decision; the structural properties the paper attributes to the journal are invariant across that choice. A commonly raised concern — that a journal is a black box for operations staff who must inspect it under pressure — is therefore largely a deployment-configuration question rather than a property of the principle.

The structural gap, and the workloads under which it appears. A `Confirm` invocation in the journal carries the parameters of one verb plus its administrative metadata — typically tens of bytes. The same operation expressed as relational mutations must enumerate every order-item bound to the event, copying parent dimensions onto every child row, because SQL's `JOIN` cannot quantify — it materializes Cartesian projections of existing rows, it does not abbreviate them. For an event with many bound items, the relational expansion exceeds the script representation by several orders of magnitude.

The gap is regime-dependent, and presenting only the favorable case would mislead. Three regimes are worth distinguishing.

Broadcast operations over large aggregates (the worked example above): one verb expressed as the framework's script applies to N items via the receiver's

implicit quantification; the relational form must enumerate N rows. The write-amplification ratio is roughly N. The structural gap is large — several orders of magnitude when N is in the thousands — and grows with aggregate cardinality. Events bound to many order-items, role permissions broadcast to large user populations, and price changes applied across product catalogues all fall here.

Pointwise operations (one verb, one row affected): the journal grows by one script entry; the relational form mutates one row. The write-amplification ratio is roughly 1. The structural gap on storage volume is absent. The information density per entry is still higher under the operation-centric form (the entry records the script, not the received DTO), but the storage footprint is comparable. Pointwise updates — credit one wallet, increment one counter, mark one item shipped — fall here.

Analytical workload (read-heavy, snapshot queries): the journal does not address the workload directly; analytical queries run against projections, not the journal itself. The journal grows monotonically with operations regardless of read demand; the relational projection sized to the analytical workload occupies a stable footprint determined by current state. On *storage volume*, the journal accumulates without bound while the projection does not — the ratio inverts and favors relational storage. The trade is paid in journal storage (which §4.3 and Paper 5 argue is operationally cheap and write-only) and in projection-replay latency for cold projections; the gain is the ability to rebuild any projection against evolved semantics. Whether the trade is favorable depends on workload composition and on whether projection-rebuild is a valued operational capability.

The structural gap is therefore *strongest under broadcast regimes*, *neutral under pointwise regimes*, and *inverts in storage volume under purely analytical workloads*. Information density per entry remains higher under operation-centric encoding in all three regimes; the storage-volume gap is regime-specific.

Why the gap is structural. The compactness of the script representation rests on three model properties:

- **Quantification.** The receiver implicitly ranges over the affected aggregate; the verb applies to all members at once.
- **Polymorphism.** A single `Settle(outcome, ...)` accepts heterogeneous outcomes as a sum type, sharing representation across branches.
- **Parameters as metadata.** Operator, date, and outcome travel as arguments of one statement; in the relational form they are copied into every affected row, because the row is the unit of identity.

The relational form has none of these. A `JOIN` cannot quantify universally; an `UPDATE … WHERE …` issues the directive, but its result is enumerated rows. The porosity of the relational layer denotes here, in concrete terms, the structural consequence of substituting *“ item event”* with *“N copies of the antecedent.”*

Forensic observation. The diagnostic is not specific to this example. Any mature relational schema can be read for porosity directly: counting a verb's param-

eters against the columns of the rows its invocation affects, identifying NULL-allowed fields whose values the originating operation never had to supply, and noting cross-product tables that materialize what the verb expressed as quantification. Appendix B applies this procedure in detail to the *Ordering* bounded context of Microsoft's `dotnet-eShop`, a reference architecture constructed independently of the present authors, and reports specific findings against its EF Core schema. Under the regime distinctions above, the eShop case exhibits horizontal sparsity in the domain layer (per-state populated columns) and vertical sparsity at the persistence layer (`Ignore(b => b.DomainEvents)`), both substrate-induced; the write-amplification ratio of the workload is left as an open question, since the analysis is structural rather than measured.

Existence proof in production. A system whose architecture jointly satisfies the three conditions of §3 has been in continuous production use since 2018, after more than a decade of preceding refinement (described in §4.0) traceable to a 2005 prototype. Puppeteer, the framework described in §4, currently runs structurally distinct subsystems within the core of eCommerce and payments platforms: payment hubs, account-balance ledgers, KYC pipelines, customer-facing storefronts and experiences, and payment-processor integrations. Code references throughout this paper (Appendix A) point to verifiable mechanism implementations. A separate case-study paper presents quantitative observations from these deployments — endpoint latency distributions, journal growth rates, replay performance, and developer-velocity comparisons — alongside the operational details (deployment, workload, infrastructure) that fall outside the scope of an analytic paper.

What the existence proof does and does not establish. Three limitations are worth naming. *First, retrospective interpretation.* The construct *porosity* was named after the system existed (§4.0); the system therefore demonstrates that the three conditions are *jointly satisfiable in production*, not that the frame *explains* its operational success. *Second, single-realization risk.* The inverse realization is currently coextensive with Puppeteer. Appendix B reduces this risk by applying the porosity diagnostic to a public reference schema (Microsoft's `dotnet-eShop`) constructed independently of the present authors — but Appendix B analyzes a *porous* system, not an *anti-porous* one constructed by an independent party. *Third, falsifiability of the principle as such.* The analytic frame predicts that operation-centric encoding admits density preservation; it does not currently predict outcomes the system would falsify. Specifically: a system that operates well while violating the principle would show the conditions are not necessary; a system that satisfies the conditions and fails would show they are not sufficient. We have not identified instances of either.

Open invitation. Three forms of external evidence would strengthen the analytic claim and are explicitly invited. *(a)* Analyses of independently-constructed systems under the porosity diagnostic (analogous to Appendix B), particularly systems built before 2024 that satisfy operation-centric encoding through different mechanisms — Akka Persistence, Microsoft Orleans + EventStore,

or domain-specific event-sourced systems would be natural candidates. *(b)* Independently-constructed systems that satisfy the three conditions of §3 and fail under operational pressure, which would falsify sufficiency. *(c)* Independently-constructed systems that operate well at scale while explicitly violating the three conditions, which would falsify necessity. The author maintains a companion repository where such analyses are linked alongside the present paper.

Note on evaluation deferral. Deferring quantitative measurements to a companion case-study is a pattern recurrent across the paper series. Under Gregor Type I the deferral is supported — the analytic frame stands on independent grounds — but each analytic paper has so far transferred its empirical debt forward, with no paper yet redeeming the full debt. Three statements bound the position.

What is already published. Companion Paper 5 (*Substrate operations*) includes labs L1–L6 reporting append latency, density-per-event, cross-DC convergence parity, and buffer/RDBMS comparisons — bounded empirical observations scoped to Paper 5's structural claims about substrate primitives, not a comprehensive case-study of Puppeteer's production deployment.

What the present paper does not claim. Claims about Puppeteer's holistic operational performance — endpoint latency distributions, developer-velocity comparisons, replay performance, longitudinal journal growth — are not made here. The present paper invokes the system only for the structural observations §4 lists and the existence claim of §5. Subsequent analytic papers may, like Paper 5, report bounded empirical observations scoped to their own structural claims; the comprehensive case-study is separate.

What is owed. The comprehensive case-study paper is owed; the analytic series will not continue indefinitely deferring it. A reader who finds the analytic series progressing while the case-study remains unwritten is observing the failure mode the present authors name here and commit to avoid.

6. Counter-arguments

This section addresses the strongest objections to the argument advanced in §§1–5. Each objection is presented in its strongest form before its rebuttal; rebuttals draw on the principle articulated in §§1–3, not on the specific realization of §4.

"This is just CQRS done well." CQRS already separates read and write; what does anti-porosity add? Two things, addressed in Claim 5 and §3. First, CQRS is one of four traditions that document a face of structure-centric encoding locally; anti-porosity is the unified diagnosis of structure-centric encoding as the common defect. CQRS implementations vary widely in this respect — some recreate porous DTOs and porous event payloads despite separating reads from writes. Second, the contribution of this paper is not any pattern (CQRS or otherwise) but the recognition that the four traditions converge on a single

architectural choice — structure as the representational primary — which prior work has not named with sufficient generality to see across surfaces. **CQRS addresses separation of concerns; anti-porosity addresses the encoding decision (what the representation is shaped for). The two operate at different conceptual layers.**

"In small systems, porosity doesn't hurt." True. The operational costs of porosity scale with system size, longevity, and rate of change. The conceptual claim, however, does not depend on these. Porosity is a structural property visible at any scale; whether one chooses to address it depends on the regime in which the system operates. This paper does not advocate adopting any specific framework for small or short-lived systems; *When NOT to use this approach* explicitly enumerates regimes where the realization pattern is not the right fit, and the principle itself remains diagnostic in those regimes — porosity remains a real pathology, even where its correction would not repay its cost. **The claim of this paper is therefore diagnostic rather than prescriptive: it names a defect whose relevance depends on scale, not one whose correction is universally mandatory.**

"Joins are sometimes necessary." Anti-porosity does not prohibit joins. **As illustrated in §5, joins migrate to the read side, where enumeration serves analytical needs rather than polluting the representation of operations.** The unified principle (§3) constrains only how state is *recorded* — not how it is queried for derivative purposes. Read projections may be denormalized, joined, indexed, and aggregated however a downstream consumer requires; the journal remains dense, but analytical and reporting use cases retain the full vocabulary of relational queries against derived views.

How read projections are constructed under anti-porosity. Two mechanisms are available, both operation-centric. First, *replay-on-demand*: a consumer requests a projection by replaying the journal against a projection-specific reducer. This is correct, evolution-tolerant (the reducer can be updated and the projection rebuilt), and cheap for projections that aggregate small subsets of the journal. Second, *incremental projection via reactions*: a long-lived observer subscribes to the operation stream and updates a projection store as each verb commits. The observer is described as a *Reaction* in the companion Paper 3; it is itself operation-centric — the reaction processes the same operation that the journal records, not a relational diff of state — and therefore does not reintroduce porosity at the source-of-truth boundary. Projections may be relational (typed rows, indexed for query patterns) without polluting the journal, because the relational projection is *derived* from operations rather than *defining* the source of truth. The cost paid for relational reads is conventional (denormalization, indexing, eventual consistency lag), not structural; under anti-porosity the relational projection is one consumer of the source of truth, not its custodian. The full treatment of reactions, their consistency model, and projection rebuild strategies is the subject of Paper 3.

"Business intelligence and analytics require SQL." Conventional BI as-

sumes that the relational store *is* the source of truth. Under anti-porosity, the operation log is the source of truth, and analytical projections are downstream consumers. SQL access remains available — to the projection, not to the source of truth. **Because the operation log stores operations rather than state (§3, §4.3), projections for BI can be recomputed with evolving semantics — a capability unavailable when the relational store is itself the historical record.**

"Our data lake is the source of truth." This is an organizational decision, not a structural property of the system. The data lake's role can be reframed: it remains a shared projection consumed by downstream teams, while the operation log is the source of truth for the producer system. Anti-porosity is preserved internally even when external contracts demand denormalized projections at the system boundary. **The distinction is between organizational source of truth and architectural source of truth. Anti-porosity concerns the latter.**

7. Related work

This paper sits at the intersection of multiple bodies of work. Two prior streams attempted cross-tradition unification — *object-relational impedance mismatch* and the *data abstraction* tradition — and are reviewed first; four further traditions addressed faces of porosity locally and are reviewed after. Each is positioned for its relationship to the present paper, not for completeness; the literature review is not exhaustive (the methodological caveat is stated explicitly in §7.x below).

Prior unification attempts. The diagnosis that database-shaped representations and behavior-shaped objects do not align cleanly has a long history under the name *object-relational impedance mismatch*. Ireland, Bowers, Newton, and Waugh (2009) provide the canonical peer-reviewed treatment: a systematic classification of the mismatch's faces along structural, behavioral, and architectural axes, with explicit comparison of the mapping strategies (ORM, OODB, document-relational) that have been proposed against each face. That paper is the closest prior naming to the present paper's *porosity*: both describe the cost of recording objects under a relational primary, both diagnose the cost (NULLs, joins, fragmented hierarchies) as structural rather than incidental, and both observe that the proposed remedies — work *within* the relational paradigm, do not question it.

The present paper differs from impedance-mismatch literature on three points. First, the impedance-mismatch frame is binary — object vs relational — and does not generalize to surfaces where neither side of the binary applies: an endpoint contract is not "object" or "relational", an event-sourcing payload is neither either, yet both exhibit the same defect under the encoding-decision frame of §1.1. Second, impedance-mismatch literature names the *layer* at which the mismatch occurs (the object/relational boundary) without naming the encoding

decision underneath it (structure as the representational primary); once that decision is named, the same defect becomes visible at endpoint and event-sourcing layers where no relational substrate is present. Third, impedance-mismatch remedies historically work *within* the relational paradigm (ORMs, document mappers, polyglot persistence, schema-on-read) rather than questioning structure as primary; the present paper proposes the inverse choice (operation as primary) projected across surfaces.

The *data abstraction* tradition (Parnas, 1972; Liskov & Zilles, 1974) and its successors in module-level encapsulation address a related but distinct concern: hiding implementation behind operations exposed by an interface, so that the representation can change without callers noticing. The principle "the representation should fit the operations that act on it" is older than the present paper's diagnosis, and to that extent the present paper is descended from this tradition rather than novel against it. What this paper adds is the observation that the principle is honored at the module/class boundary but routinely violated at the persistence, contract, and event-payload boundaries — surfaces that the data-abstraction literature did not theorize as needing the same discipline. The contribution is the projection of the data-abstraction principle to surfaces where it had not been previously claimed, plus the unification with impedance mismatch under a single encoding-decision frame.

Algebraic data types and type theory. The sum-type vs product-type framing used throughout this paper (§2.1, §3) is borrowed directly from algebraic data type theory (Hindley, 1969; Milner, 1978), where the distinction has been formally established for decades. The contribution of the present paper is not this framing itself but its application to runtime representation: *which encoding the representational primary adopts at each layer* is the architectural decision the four traditions of §2 each documented in their own vocabulary. Type theory provides the formal language; the paper applies it as an architectural diagnostic across surfaces that compile-time type systems do not reach.

Operation-based CRDTs. In distributed-systems literature, operation-based CRDTs (Shapiro, Preguiça, Baquero, & Zawirski, 2011) privilege operation over state in their replication semantics: the durable artifact replicated between nodes is the operation, not the resulting state. This is operation-centric encoding at the replication layer, established and well-defined. The present paper's contribution is the cross-surface generalization: the same encoding decision can be made at the persistence primary, the domain primary, and the endpoint primary of a single-node system, not only at the replication boundary between nodes. The unification is across surfaces, not within the replication layer alone.

Append-only architectures. Append-only durable artifacts appear in Datomic, Kafka with log compaction, and the broader event-sourcing tradition. These systems share the structural property — additions only, no in-place mutation — but differ from the present realization in *what is appended*: append-only databases and logs typically append serialized state-deltas or command DTOs, not executable scripts of the operation that consumed

specific inputs (§4.3). Append-only is therefore necessary but not sufficient for operation-centric encoding; the latter requires the additional decision that what is appended *is the script*, not the DTO.

Four applied traditions. *Domain-driven design* (Evans, 2003; Vernon, 2013) documents *anemic domain models* and *persistence-driven design* as pathologies and prescribes ubiquitous language and aggregate-rooted modeling as remedies. It diagnoses the symptom — a domain that lacks behavior — without naming the encoding decision behind it (structure-centric domain representation in service of structure-centric persistence). *REST API design* (Fielding, 2000; Richardson and Ruby, 2008) and the GraphQL specification (2015) document overloaded endpoints and fixed-shape DTOs, prescribing resource orientation and field selection respectively. GraphQL field selection moves toward operation-centric encoding at the wire — each invocation expresses what it consumes, not the union — but the discipline lacks corresponding operation-centric expression in the domain or persistence layers. *Event Sourcing* (Young, 2010; Fowler, 2005; Vernon, 2013) documents event-payload bloat and command-event coupling and prescribes command-event separation, upcasting, and versioning; it recognizes that persisting state is insufficient and persists operations instead, but typically represents those operations as fixed-shape command DTOs — the envelope of what was received rather than the script of what executed — a partial inversion that does not complete to operation-centric encoding. *Database theory* (Codd, 1970; Kent, 1983; Fagin, 1977) is the oldest and most formal of the four; it documents normalization anomalies and NULL semantics and prescribes successive normal forms. It names the effects (anomalies) without questioning the underlying decision: the relational model records state as structure, never operations as primary.

These four traditions have rarely engaged with each other on the question of representational form, perhaps because their domains appear to address different problems. This paper argues that they are observing the same encoding decision through different lenses. Each tradition named a face of structure-centric encoding in its own layer and prescribed remedies that work within the structure-centric paradigm rather than questioning it. Among the cross-tradition unifications already reviewed, impedance-mismatch literature covers one surface (objects/relational) and the data-abstraction tradition addresses representation-fitting-operations at the module level; neither, to our knowledge, names the encoding decision (structure vs operation as the representational primary) at the cross-surface level proposed here.

Four formalisms. The grounding established in §1.1 invokes existing formal vocabulary, with two abstractions introduced here. Graph theory (Diestel, 2017), with extension to typed and labeled graphs in the Semantic Web (Lassila & Swick, 1999) and graph databases (Robinson, Webber, and Eifrem, 2015), provides the language of semantic graphs. Information theory (Shannon, 1948) inspires — without formally underwriting — the framing of representations in

terms of *structural capacity* and *informational content*; both terms are this paper's, not Shannon's. Storage-engine literature (Hellerstein, Stonebraker, and Hamilton, 2007) catalogues the relational, document, and log substrates whose properties we abstract under the term *structural alphabet*; that abstraction is introduced here. Database normalization theory (Codd, 1970, 1972; Fagin, 1977) provides the canonical historical formalism for the relational case. The contribution is the observation that these four already-established formalisms — together with the two bridging terms — agree on a single defect that none names with sufficient generality across substrates. **The paper's formal vocabulary is therefore mostly assembled and partly introduced — explicitly so where introduced.**

Aspect-oriented programming and ORM annotations. The aspect-oriented programming tradition (Kiczales et al., 1997; AspectJ; Spring AOP) identified persistence as one of the ugliest cross-cutting concerns and sought to externalize it from domain code. The most widespread practical instantiation is ORM-style annotations (the Java Persistence API, Hibernate, Entity Framework). These approaches share a goal with anti-porosity — keeping the domain class free of persistence concerns — but operate at a different level: they decorate the domain class with framework-specific metadata that describes how to map its fields to a relational substrate. The substrate's structural alphabet is preserved; only the syntactic appearance of pollution is moved from the class body to its annotations. Anti-porosity, by contrast, addresses the substrate itself: by replacing relational projections with an executable journal of operations (§4.3), the question of how to map domain fields to relational columns disappears. **The domain class need not be annotated, because the persistence substrate no longer requires a projection of its fields.**

Other actor-model frameworks. Multiple frameworks implement combinations of CQRS, the Actor Model, and Event Sourcing. Akka (Lightbend) on the JVM, Microsoft Orleans (Bernstein et al., 2014) with its grain abstraction, Proto.Actor across multiple languages, and the dedicated EventStore database all share substantial conceptual infrastructure with the realization presented in §4. They differ from it, and from one another, in a single consistent respect: **each persists events as serialized data structures defined at the command boundary, rather than as executable scripts derived after parameter evaluation**. The underlying storage organization typically retains relational patterns — category-indexed streams, table-backed projections — rather than per-actor append-only structures. None currently articulates anti-porosity as a unified principle, and none persists scripts in the script-form sense developed in §4.3. The differences are not failures of these frameworks against their stated goals — each does what it sets out to do — but they illustrate that anti-porosity requires a deliberate decision at the substrate layer that mainstream actor frameworks have not made. The decision is available to them: any of these frameworks could be extended to record operations as scripts rather than as DTOs, and to migrate the storage organization from category-indexed streams to per-actor append-only journals. **The distinction is not in the**

use of actors, CQRS, or event sourcing, but in what is chosen as the durable representational unit.

Scope of the literature review. The review above is positioned rather than exhaustive. We did not conduct a systematic literature review in the PRISMA sense: we did not enumerate ACM DL, IEEE Xplore, or dblp queries with predeclared keywords and inclusion criteria. The traditions reviewed were selected because they are the ones in which the present authors have prior fluency and whose vocabulary the paper engages with directly. Other prior naming attempts may exist that the present authors are unaware of; the falsifiable form of the paper's novelty claim is therefore: *"we are aware of no prior work that names structure-vs-operation as the representational primary at the cross-surface level proposed here, and we engage explicitly with the two cross-tradition unifications we did find (impedance mismatch, data abstraction)"*. A reader who knows of a prior naming the paper missed is invited to write to the author; the contribution will be revised accordingly.

What this paper adds. The paper does not propose a new pattern. It identifies the encoding decision (structure vs operation as the representational primary) that the four applied traditions each described in surface vocabulary; positions the contribution as an *extension* of impedance-mismatch literature (covering surfaces beyond the object/relational boundary) and of the data-abstraction tradition (projecting its principle to persistence, contract, and event-payload surfaces); names the defect (*porosity*) and its principled inversion (*anti-porosity*); and demonstrates that the inversion is implementable as a unified discipline across surfaces. The Puppeteer framework serves as proof-of-existence. The analytic frame is *in principle* independent of any specific realization, but in practice the only realization currently known to the authors that jointly satisfies the three conditions is Puppeteer; the §7 review of "Other actor-model frameworks" enumerates frameworks that could be extended to satisfy them but do not currently. Independence of the frame from any specific realization is therefore *conjectural*, not established; the open invitation in §5 is the explicit mechanism by which the conjecture would be tested. **The novelty lies in naming the encoding decision at a level of generality the prior unifications did not reach, and in showing that the inversion is operationally realizable in at least one production system.**

8. Conclusion

This paper has identified the encoding decision — *structure vs operation as the representational primary* — that four applied traditions each documented in surface vocabulary, and that prior cross-tradition unifications (impedance mismatch, data abstraction) named at narrower scope. We have named the structure-centric defect *porosity*, and the operation-centric inversion *anti-porosity*. Anti-porosity requires three simultaneous conditions: a domain whose hierarchy admits sum-type structure natively, an endpoint whose contract permits per-call selectivity, and a persistence layer that records the script that

executed rather than the state that resulted or the command that was received. A system that satisfies all three through a single architectural decision — the executable journal — has been in continuous production use since 2018, after more than a decade of preceding refinement traceable to a 2005 prototype (§4.0); that system, Puppeteer, is presented in §4 as the existence proof for the principle, not as its definition. The analytic frame is *in principle* independent of any specific realization, but Puppeteer is currently the only realization we know of that jointly satisfies the three conditions; whether other systems could realize them through different mechanisms is open to test (§5 *Open invitation*). In the vocabulary established in §1.1, anti-porosity aligns structural capacity with informational content across the three representational layers a system exposes.

The implications extend beyond the paper. When porosity is removed at the substrate, much of the infrastructure traditionally introduced to compensate for substrate mismatch becomes optional rather than essential: caches that hid slow joins, ORMs that translated between domain and table, message queues that moved flat tuples between systems. The question of how systems built on porous substrates may adopt the new discipline progressively follows from the autopersistence property the framework inherits from its 2005 origin: a system whose initial state and event sequence are observable can be reconstructed independently of how it persists internally; a system whose only durable artifact is state cannot reconstruct the operations that produced it.

Several lines of work follow. A comprehensive case-study paper will report quantitative measurements from the framework's production use; bounded empirical observations supporting specific structural claims have already appeared in Paper 5 (labs L1–L6) and are referenced in §5 of the present paper. Six companion papers extend the present argument across the series (Papers 2–7), enumerated in the Cross-references section below. Independently, the forensic procedure introduced in §5 invites verification against public corpora — an exercise initiated in Appendix B against `dotnet-eShop` and open to readers for replication in their own domains.

The contribution of this paper is the identification of the encoding decision as the common cause behind four locally-named surface manifestations, the analytic frame that allows the decision to be diagnosed across surfaces (§§1–3), the demonstration that the inversion is realizable in production through one existence proof (§4) and that the diagnostic reproduces in a system the present authors did not build (Appendix B), and the explicit invitation for external falsification (§5 *Open invitation*). Whether the construct survives broader scrutiny depends on what readers find when they apply the diagnostic in their own domains and on whether independently-constructed systems satisfying or violating the three conditions of §3 come to light. Naming the encoding decision is a precondition for these tests, not a substitute for them.

Cross-references

This paper establishes vocabulary that subsequent papers in the series reuse:

- *Paper 2 — Dual compilation*: the anti-porosity principle as it manifests at the language layer (interpreted versus compiled execution paths).
- *Paper 3 — Reactions and the partition: opt-in eventual consistency in actor-native systems*: the consistency model that anti-porosity admits, with deferred derivative behaviors expressed as domain code rather than externalized pipelines; also the mechanism by which read projections are constructed under anti-porosity (§6).
- *Paper 4 — Preserving semantic continuity across actors: a tell-based approach without orchestration*: cross-actor coordination under operation-centric encoding; the journal's density is prerequisite for the handoff mechanism.
- *Paper 5 — The Journal as Substrate: Unifying Deployment, Replication, Backup, and Offline Operation in Distributed Systems*: the substrate operations enabled by the executable journal; reports labs L1–L6 that empirically support its structural claims (also referenced from §5 of the present paper for the evaluation-deferral discussion).
- *Paper 6 — Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks*: extends the anti-porosity argument outward to the broader infrastructure ecosystem, treating compensating layers (caches, ORMs, message queues) as artifacts whose necessity dissolves once the substrate is changed.
- *Paper 7 — After the substrate: building software without a datacenter*: extrapolates the argument forward, examining what is possible once journal-as-substrate is the operational primary rather than a substitute for relational persistence.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `b42d0f7` (2026-05-26), which ports the SimpleX/StageManager/Usher fixes from the active development branch on top of the prior public snapshot. The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:177e2d61486bdbdfd2d5e774fcf392a45406e60d;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:b42d0f76b4278c36a45c221cc1453e3ee8ffb3de
```

A prior snapshot — commit `2f31f96` (2026-05-18) — is also archived in Software Heritage and remains resolvable for reference:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
```

`anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da`

The line numbers cited in the body have been verified against `b42d0f7` and remain stable across the two snapshots; the SimpleX integration did not move the cited mechanisms.

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:846`) resolve against the `b42d0f7` snapshot. A reader can construct per-file SWHIDs by adding the qualifiers `;path=<path>;lines=<NN>` to the `b42d0f7` directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Bernstein, P., Bykov, S., Geller, A., Kliot, G., & Thelin, J. (2014). *Orleans: Distributed virtual actors for programmability and scalability* (Microsoft Research Technical Report MSR-TR-2014-41). Microsoft Research.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.

Codd, E. F. (1972). Further normalization of the data base relational model. In R. Rustin (Ed.), *Data base systems* (Courant Computer Science Symposia 6, pp. 33–64). Prentice-Hall.

Diestel, R. (2017). *Graph theory* (5th ed.). Springer.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fagin, R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM Transactions on Database Systems*, 2(3), 262–278.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Fowler, M. (2005). *Event sourcing*. martinfowler.com. https://martinfowler.com/eaaDev/EventSourcing.html

GraphQL Foundation. (2015). *GraphQL specification*. https://spec.graphql.org

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.

Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a database system. *Foundations and Trends in Databases*, 1(2), 141–259.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

Hindley, R. (1969). The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society*, 146, 29–60.

Ireland, C., Bowers, D., Newton, M., & Waugh, K. (2009). A classification of object-relational impedance mismatch. In *2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA '09)* (pp. 36–43). IEEE.

Kent, W. (1983). A simple guide to five normal forms in relational database theory. *Communications of the ACM*, 26(2), 120–125.

Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C., Loingtier, J.-M., & Irwin, J. (1997). Aspect-oriented programming. In M. Akşit & S. Matsuoka (Eds.), *ECOOP'97 — Object-oriented programming* (pp. 220–242). Springer.

Lassila, O., & Swick, R. R. (1999). *Resource description framework (RDF) model and syntax specification* (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/1999/REC-rdf-syntax-19990222/

Liskov, B., & Zilles, S. (1974). Programming with abstract data types. *ACM SIGPLAN Notices*, 9(4), 50–59.

McCarthy, J. (1960). Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4), 184–195.

Milner, R. (1978). A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3), 348–375.

Mooers, C. N., & Deutsch, L. P. (1965). TRAC, a procedure-describing language for the reactive typewriter. In *Proceedings of the 20th National Conference of the ACM* (pp. 229–246).

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058.

Richardson, L., & Ruby, S. (2008). *RESTful web services*. O'Reilly Media.

Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases* (2nd ed.). O'Reilly Media.

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423; 27(4), 623–656.

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)* (pp. 386–400). Springer.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Young, G. (2010). *CQRS documents*. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

---

Appendix A: Source-code verification

This appendix consolidates the source-code references cited throughout the paper. All paths are relative to the public Puppeteer repository; line numbers reflect the source at commit `b42d0f7` (see *Code provenance* above). The framework's source was migrated from Spanish to English identifiers during paper development; the line numbers and file names below reflect the English-language source as it currently stands.

A.1 Reflection-based discovery (§4.1)

Claim	File	Lines
`Actor` marker base class for domain entry points	`Puppeteer/Actor.cs`	15
Reflection-based discovery of domain types via assembly scan	`Puppeteer/EventSourcing/DomainLibraries.cs`	117–128
Domain library loaded at handler construction	`Puppeteer/EventSourcing/ActorHandler.cs`	61
Polymorphic argument-parameter compatibility at parse time (`AreCompatible`)	`Puppeteer/EventSourcing/Interpreter/Libraries/AstExpression.cs`	236–246
Runtime substitution of subtypes via symbol table (`IsSubclassOf` check)	`Puppeteer/EventSourcing/Interpreter/SymbolTable.cs`	134

A.2 Parameter modifiers (§4.2)

Claim	File	Lines
Four parameter modifiers defined (`In`, `Out`, `InOut`, `Eval`)	`Puppeteer/Parameter.cs`	33–36
`Eval` value setter generates literal-assigning script on first invocation	`Puppeteer/Parameter.cs`	163–224
`EvalScript` property (valid only for `Eval` parameters)	`Puppeteer/Parameter.cs`	228–247
Parser recognises `Eval` modifier in parameter declarations	`Puppeteer/Parameters.cs`	65, 110–111
`Eval` parameters' `EvalScript` serialized to journal	`Puppeteer/Parameters.cs`	334–367
`Out` parameters serialize as `?` placeholder	`Puppeteer/Parameters.cs`	755–757
`?` for `Out` parameters deserializes to default value	`Puppeteer/Parameters.cs`	780–782
`?` tokenised as `TokenType.question` in the lexer	`Puppeteer/EventSourcing/Interpreter/Lexer.cs`	600
V1 `eval` statement preserved (interpreted only); V2 redirects to parameter modifier	`Puppeteer/EventSourcing/Interpreter/Libraries/EvalStatement.cs`	52–55

A.3 Executable journal (§4.3)

Claim	File	Lines
Journal high-level interface (append-style writes, read-forward only)	`Puppeteer/EventSourcing/DB/Diary.cs`	228–337
Append primitive (`AppendRecord`)	`Puppeteer/EventSourcing/DB/FileSystem/JournalWriter.cs`	65
Read-forward primitive (`ReadAll`)	`Puppeteer/EventSourcing/DB/FileSystem/JournalReader.cs`	35

Claim	File	Lines
Script entries persisted as UTF-8 bytes (`EncodeScriptEvent`)	`Puppeteer/EventSourcing/DB/FileSystem/BinaryEventCodec.cs`	41–101
`ReplayEvent` re-executes scripts at journal replay	`Puppeteer/EventSourcing/ActorHandler.cs`	846
On-demand parser invocation for type resolution at replay	`Puppeteer/EventSourcing/Follower/Reaction.cs`	1027–1029

Appendix B: Diagnostic application to an independently-constructed schema

B.1 Purpose

This appendix applies the diagnostic procedure of §5 ("Forensic observation") to a schema constructed independently of the present authors. The purpose is to test, in the smallest available form, whether the construct *porosity* is identifiable in a system whose authors had no relationship to the construct's naming. A positive identification reduces — though does not eliminate — the risk named in §5 that the construct is coextensive with the present authors' realization.

The system selected is the *Ordering* bounded context of Microsoft's `dotnet-eShop` (https://github.com/dotnet/eShop), the official .NET reference implementation for cloud-native e-commerce. The Ordering context implements the canonical DDD + CQRS + event-sourcing patterns that §2 and §7 of this paper engage with. References below resolve against commit `9b4f943` (2026-current upstream `main`).

B.2 Method

The diagnostic of §5 asks three questions of any persisted representation:

1. **Horizontal sparsity (§2.1):** are columns of a table populated only by some rows, with the discriminator silently encoded in another column?
2. **Vertical sparsity (§2.3a):** are domain operations recorded as persistent artifacts, or only their terminal effects on state?
3. **Operation vs row (§5):** where the substrate forces enumeration of rows, would an operation-centric representation quantify (one verb over an aggregate) rather than enumerate?

We apply each question to the schema established by the initial EF Core migration plus its 2023–2024 refinements.

B.3 Observations

Schema under analysis. Migration `20230925222426_Initial.cs` (lines 122–162) creates the `ordering.orders` table with eleven columns:

Column	Nullability	Set by
`Id`	NOT NULL	sequence `orderseq`
`Address_Street`, `_City`, `_State`, `_Country`, `_ZipCode`	NULL	Address value object (owned entity flattened to 5 columns)
`OrderStatusId`	NOT NULL	enum discriminator
`Description`	NULL	varies by state transition (see below)
`BuyerId`	NULL	set by `SetPaymentMethodVerified()`, not at construction
`OrderDate`	NOT NULL	constructor
`PaymentMethodId`	NULL	set by `SetPaymentMethodVerified()`

Horizontal sparsity test (§2.1). The `OrderStatus` lifecycle (`Ordering.Domain/AggregatesModel/OrderAg 14`) defines six states: `Submitted`, `AwaitingValidation`, `StockConfirmed`, `Paid`, `Shipped`, `Cancelled`. The schema collapses all six into a single discriminator column (`OrderStatusId`). The `Description` field is the clearest horizontal-sparsity case: its meaning is conditioned on `OrderStatusId`, and its values are assigned literally as the lifecycle advances (`Order.cs:115,126,138,151`):

`OrderStatusId`	`Description`
`Submitted`	NULL
`AwaitingValidation`	NULL
`StockConfirmed`	`"All the items were confirmed with available stock."`
`Paid`	`"The payment was performed at a simulated \"American Bank checking bank account ending on XX35071\""`
`Shipped`	`"The order was shipped."`
`Cancelled`	variable per cancel path

The `Description` field is the operation's residue, persisted into a structural column whose meaning the schema does not encode and whose population is condi-

tional on a sibling column. This is the §2.1 defect literally — a sum-type discriminated field encoded as a product-type column. `BuyerId` and `PaymentMethodId` are nullable for the same reason: they are determined later in the lifecycle by `SetPaymentMethodVerified()`; the schema accepts orders that have not yet been verified by populating NULL.

An anecdotal aside, not part of the diagnostic. The domain class `Order` (line 23) carries a private boolean field `_isDraft` accompanied by `#pragma warning disable CS0414 // The field 'Order._isDraft' is assigned but its value is never used`. This is in-memory dead code — plausibly a minor maintenance artifact rather than evidence of structural porosity, and we explicitly do not count it as diagnostic. We mention it because the analogous pattern at the persistence layer would be diagnostic — columns retained across migrations whose values no operation reads — but the present analysis does not enumerate such columns in the schema and the inference from one to the other is suggestive, not probative. Readers should weight the in-memory observation accordingly.

Vertical sparsity test (§2.3a). The Entity Framework configuration (`Ordering.Infrastructure/EntityConfigurations/OrderEntityTypeConfiguration.cs:9`) declares `orderConfiguration.Ignore(b => b.DomainEvents)`. The aggregate's domain operations — `OrderStartedDomainEvent`, `OrderStatusChangedToAwaitingValidationDomainE` `OrderStatusChangedToStockConfirmedDomainEvent`, `OrderStatusChangedToPaidDomainEvent`, `OrderShippedDomainEvent`, `OrderCancelledDomainEvent` — are dispatched in memory and excluded from persistence by construction. The schema captures the terminal state (`OrderStatusId = Shipped`), not the operations that produced it. The transition trace exists only ephemerally, during the request that produces it, and is then dropped. This is the §2.3a defect literally: state-as-form persisted, operation-as-trace discarded.

Outbox as §2.3b instance. Migration `20231026091055_Outbox.cs:14-30` adds an `IntegrationEventLog` table:

Column	Type
`EventId`	uuid
`EventTypeName`	text
`State`	int
`TimesSent`	int
`CreationTime`	timestamp
`Content`	text (JSON-serialized payload)
`TransactionId`	uuid

The `Content` column holds the integration event serialized verbatim — the command DTO as received, not the script of what executed. This is the §2.3b defect at the outbox boundary: the event payload is shaped by the type-space

of any integration event the system may emit, not by the specific operation that produced this event. Additionally, the `State` column itself reproduces the §2.1 defect — the outbox event's own lifecycle (`Pending`, `InProgress`, `Published`, etc.) collapsed into a discriminator column.

Operation vs row test. The `SetStockConfirmedStatus()` operation (`Order.cs:108-117`) is, semantically, one verb applied to the aggregate. Under the relational projection, no row in the `orders` table records that this operation occurred; only `OrderStatusId = StockConfirmed` and `Description = "All the items..."` are observable, both attributes of the terminal state. To answer the question *"at what time did this order transition to StockConfirmed?"*, the schema has no answer — the operation's timestamp is not in the schema (only `OrderDate`, the constructor time, is recorded). To recover the transition history one must join against the dispatched (but unpersisted) domain events, which is impossible after the request that produced them returns. The operation existed; the row does not record it. An operation-centric representation would quantify (one verb in the journal: `SetStockConfirmed(orderId, at: timestamp)`); the relational form has no quantifier and no place to put the verb.

B.4 What this demonstrates

The diagnostic reproduces. The Ordering bounded context of `dotnet-eShop` — constructed by Microsoft engineers without reference to the construct *porosity* — exhibits all three of the manifestations enumerated in §2: horizontal sparsity in `orders.Description` (§2.1), vertical sparsity by explicit `Ignore(b => b.DomainEvents)` (§2.3a), and contract-induced sparsity at the `IntegrationEventLog.Content` blob (§2.3b). The construct is therefore not strictly coextensive with the present realization: its diagnostic identifies the defect in a system the construct's authors did not build.

B.5 What this does not establish

Three caveats are worth naming.

First, this is a *forensic* demonstration, not a *normative* one. The dotnet-eShop architects produced a competent reference implementation under the prevailing structure-centric default. The team's choices (single-table inheritance for `Order`'s lifecycle, owned entities for `Address`, outbox for reliable event delivery, explicit `Ignore` for in-memory domain events) are within the standard playbook for DDD + CQRS on a relational substrate. The diagnostic identifies that the system *exhibits* porosity, not that it is *wrong* by some externally-imposed standard. eShop serves its purpose; the porosity is the cost of the encoding decision the system makes, not a defect of execution within that decision.

Second, this demonstration analyzes a *porous* system, not an *anti-porous* one constructed by an independent party. The reduction in single-realization risk is therefore partial: it shows the diagnostic reproduces externally, not that the inverse encoding has been independently constructed and successfully operated.

Third, the analysis is small in scope (one bounded context of one system, one migration sequence). A systematic survey of schemas — across multiple companies and frameworks — would strengthen the claim that porosity is a default rather than a local artifact. We do not undertake such a survey here; the present demonstration is sized to the falsifiability claim of §5 (that the construct is identifiable in independently-constructed systems), not to a comprehensive empirical mapping.